

AI Trust and Situated Awareness: A Practice Theory Reframe

Monyet Batu

ORCID: 0009-0007-9002-5381

27 May 2026

*Editor's note: this essay was originally titled "AI Trust and the Meaning Layer: A Practice Theory Reframe". Subsequent essays in the series sharpened the term — what was reached for as "the meaning layer" is more precisely a form of **situated awareness** in the sense Suchman gave it in [Plans and Situated Actions](#) (1987), and the practice bundle's five elements are the captured structure of that situated awareness. Earlier Zenodo versions preserve the original "meaning layer" framing as part of the working; this revision threads the sharpened vocabulary through the text. The substantive argument is unchanged.*

A missing situated awareness

I recently watched a [video](#) on the Nate B Jones channel arguing that the real strategic layer for agent-assisted work isn't the ability to click buttons, fill forms, or make API calls — it's what he called the *meaning layer*. Three dimensions matter, the video proposes: access (can the agent reach the system), meaning (does the agent understand what the action signifies), and authority (who decides whether the action happens). Computer use solves access, but access alone isn't enough.

What he names as *meaning* is, more precisely, **situated awareness** — the practitioner's grasp, in the moment, of where they are in the work, what the goal is, what is at hand, and what would be a legitimate next step. An agent clicking a refund button doesn't have it: it doesn't know whether it's refunding from Stripe or Shopify, reversing a one-time charge or breaking a subscription, touching money or just shifting a draft status. The agent can guess, and the guesses are getting better — but for high-consequence work, what is needed is not richer types alone but richer awareness. The semantic work primitive (the refund, the reschedule, the deploy-to-production) needs to be exposed with the situated awareness a practitioner would bring to it, not hidden behind a UI the agent has to interpret.

The video goes on to propose what that better representation looks like: richer schemas, typed actions, meaning made explicit as data — so the agent can reason about it correctly. That points in the right direction but doesn't go far enough. Schemas alone are still plans-on-paper; situated awareness is the practitioner's working sense of the practice as it unfolds. Lucy Suchman made the distinction explicit in 1987: plans do not *cause* action; they are a *resource* people use while acting in situ. What the agent is missing is not the plan but the resource — the captured form of a practitioner's situated awareness about this kind of work.

Something's missing: my own experience

The points rang true against what I had been experiencing while trying to build my own harness. I had created rich infrastructure: a graph database to capture relationships, a vector database for episodic recall with access to full conversation history, integrations with health tracking (Garmin, Strava), and a news aggregation system to replace my manual morning browsing across multiple websites and email channels. I gave the AI clear function descriptions and open API specifications, exposed via the standard tools array —

a basket of typed functions the model is meant to associate with each user request, emitting a tool call when the match is confident enough. I thought I had done everything right — creating semantic richness and exposing the primitives.

But the system was brittle. Sometimes the AI would use the right function. Sometimes it wouldn't. When I made changes, things that had been working would stop working. The inconsistency wasn't about missing information — the semantics were there, well-described. It was about something deeper. The AI could read the function descriptions, but it didn't have practical sense about when and why these activities mattered, how they related to each other, or what they were actually for.

I went back to the blackboard.

Back to the blackboard

I wondered whether there was already a theory of task management out there — not specific to LLMs or AI, but rooted in the human sciences, something that described how people actually organise work. So I began with research and investigation. Which is to say: I asked an LLM.

Not passively — and this is worth a moment. Two hours earlier I had no idea practice theory existed. Two hours in, I had a working grasp of it, enough to start mapping it against a real problem. The conversation wasn't outsourced thinking; it was thinking out loud with a partner who could surface the literature, answer back when I tested my understanding in my own words, and tell me when I had something wrong — closer to Socratic dialogue than to search.

And there it was: a body of work with nearly a hundred years behind it. Practice theory — the social-theoretic tradition with roots reaching back to Wittgenstein and developed through Schatzki, Reckwitz, Shove, and others — offered a framework for exactly this problem.

Here I did do some serious reading. Carpentry was the example that kept coming up — the canonical illustration in the literature. In practice theory, carpentry names a practice: a field of work, a skillset, a recognisable bundle of activity. And the practice itself is constituted by several elements:

- Teleo-affective structures — the ends, projects, and emotional orientations that give the work its direction
- Understandings — the practical know-how that lets someone recognise what to do
- Rules — explicit instructions, codes, prescriptions
- Affordances — what the materials and tools offer up to action
- Materials — the wood, the chisels, the workbench

The crucial move is that the practice is not the carpenter. The practice is the carpentry itself — the structured field of activity. The carpenter, in the language of practice theory, is the practitioner enacting the practice.

An attempt at mapping

So how does this translate to an AI harness? My first pass:

- Teleo-affective structures — a description of the temperament and the role to be played
- Understandings — a description of the work and the tasks within it
- Rules — the rules that apply to the workflow

- Affordances — how each part of the work is done, with which tools
- Materials — the tools themselves

A stumble and a reframing

Carpentry is a learnt thing. I can't just pull someone off the street, say this is carpentry, and expect them to make me a set of shelves. A carpenter has served a long apprenticeship, learning not just what to do but how. And with that, the reframe arrived: my first attempt hadn't been a practice at all. It was a practice recipe.

The practice recipe doesn't bypass apprenticeship so much as replace it. What an apprentice would have absorbed slowly through years of participation, the LLM receives in the recipe itself: the teleo-affective structure, the rules, the affordances, the practical understandings. Apprenticeship and recipe do the same work in different media — the former in bodies and time, the latter in text and engagement.

*(Note in hindsight: this sentence uses "recipe" to name both the artifact transmitted and the act of transmission. The third essay in the series splits those — **practice bundle** for the artifact, **apprenticeship** for the means — and the more accurate restatement is: an apprentice receives the bundle through apprenticeship, in bodies and time; an LLM receives the bundle through apprenticeship, by engaging with its captured form. Same process, two media. The argument is unchanged; the vocabulary is sharpened.)*

By engaging with the recipe, the LLM is cued into the practice; in the enacting, it becomes the practitioner.

A practitioner for whom?

One thing the mapping leaves unsaid is whose practice this is. Practice theory makes the point explicit: practices are always for someone, embedded in a life. A carpenter offers their services to many, and the practice is shaped by that breadth.

An LLM is different. Each instance is enacted as a unique practitioner whose only focus is the user in front of them.

The recipe is filled in not for carpentry-in-general but for this morning, this person, this life. The result is a strange one: every user with their own private practitioner.

And the recipe doesn't stand still.

Closing the loop

A Judge practice monitors the enactment history, identifying frictions — places where the recipe and the work pulled against each other. A Smoother practice then plans and applies adjustments, folding the resolution back into the recipe itself.

The pattern is a familiar one: monitor, analyse, plan, execute, over a shared store of knowledge — the autonomic loop that the self-managing systems literature has described for decades, here cast as practices in their own right. Once the loop is set in motion, the recipe maintains itself; the human role is to bootstrap, not to govern.

A virtuous circle, running under its own power. How the loop works in detail — what the Judge attends to, what the Smoother is allowed to change, and how the bootstrap is performed — is the subject of a [follow-](#)

[up essay on the autonomic recipe loop.](#)

Practice makes perfect.

Series

This is the first of four essays in the AI Trust series:

1. **This essay.** *AI Trust and Situated Awareness: A Practice Theory Reframe* — [DOI](#).
2. *Practice Theory — The Apprenticeship and a Strange Loop* — [DOI](#).
3. *Practice Theory — The Implementation* — DOI pending Zenodo deposit.
4. *Practice Theory — A Worked Example (Calendar Stewardship)* — DOI pending Zenodo deposit.

Companion software: [practice-theory-implementation](#) — DOI pending Zenodo deposit. Source at <https://github.com/HiddenDeveloper/practice-theory-implementation>.